Python for Data Scientist

2025

Lê Thiện Hòa

Nội dung

- Pip vs. Conda vs. Uv
- Python List / Numpy / Pandas / Polars / PySpark
- Common Pitfalls in Data structure & Vectorization & Memory optimization
- Python vs. Cython vs. Rust
- Thread vs. Process, Concurrency vs. Parallelism
- Reference Counting, Race Condition, GIL & MultiThread
- MultiProcess, AsyncIO

Pip vs. Conda vs. Uv

1. pip (Python Package Installer)

Cài đặt package từ PyPI

```
pip install numpy pandas
pip list                    # xem packages đã cài
```

Lưu và tái tạo lại env

```
pip freeze > requirements.txt    # lưu dependencies
pip install -r requirements.txt  # cài lại từ file
```

Pip vs. Conda vs. Uv

1. pip (Python Package Installer)

Pip chỉ kiểm tra dependency theo từng package riêng, dễ dẫn tới xung đột khi nhiều lib yêu cầu version khác nhau.

Vd: `**pip install numpy==2.0.0 scikit-learn==1.0.2**` (với python 3.9)

- Cài đặt thành công
- Khi `import sklearn` thì lại fail

Pip vs. Conda vs. Uv

2. conda (Package & Environment Manager)

Quản lý Python library versions, environment, dependency resolution

```
conda create -n myenv python=3.11   # tạo env mới
conda activate myenv                 # kích hoạt env
conda install numpy pandas           # thêm package
```

Cấu trúc sinh ra:

- **envs/** → chứa các environment conda (nằm trong conda install)
- **environment.yml** → dùng để lưu dependency cho project
    - Export file environment: `**conda env export > environment.yml**`
    - Tái tạo environment: `**conda env create -f environment.yml**`

Pip vs. Conda vs. Uv

2. conda (Package & Environment Manager)

Quản lý Python library versions, environment, dependency resolution

Vd: `**conda install numpy==2.0.0 scikit-learn==1.0.2**` (với python 3.9)

- Cài đặt thất bại, conda **kiểm tra conflict trước khi tiến hành cài đặt**
- Fix: `**conda install numpy scikit-learn==1.0.2**`
  - Conda **tự tìm được version numpy phù hợp** với version scikit-learn muốn cài đặt
  - Pip không làm được chuyện này, nó sẽ chỉ tìm numpy version mới nhất để cài

Pip vs. Conda vs. Uv

2. conda (Package & Environment Manager)

Chạy file python: chạy bình thương `python python_script.py` sau khi đã activate môi trường

Xóa env

```
conda deactivate      # thoát env
conda remove -n myenv --all
```

Kiểm tra env đang dùng

```
conda info --envs       # danh sách env
conda list              # packages trong env hiện tại
```

Pip vs. Conda vs. Uv

3. uv (Modern Package Manager – Rust)

Quản lý môi trường & dependency resolution tương tự conda nhưng nhanh hơn nhiều (viết bằng Rust).

```
uv init .          # khởi tạo project trong thư mục hiện tại
uv add numpy       # thêm package (ghi vào pyproject.toml + uv.lock)
uv pip install x   # cài nhanh, không cần pyproject.toml
```

Cấu trúc sinh ra:

- **pyproject.toml** → khai báo dependency cần của project
- **uv.lock** → ghi đúng version của tất cả package + dependency con để có thể tái tạo lại môi trường
- **.venv/** → virtual env

Chạy: `**uv run python_script.py**` trong thư mục đã chạy `uv init`, không cần activate môi trường

Xóa env / project: xóa 3 files & folder trên

Data Structures & Libraries Benchmark - **Python List**

```
nums = list(range(10_000_000))
squared = [x**2 for x in nums]
```

- Ưu: built-in, dễ dùng.
- Nhược: loop chậm, memory overhead lớn.

Data Structures & Libraries Benchmark - Numpy

```
import numpy as np
arr = np.arange(10_000_000)
squared = arr ** 2
```

- Mảng **C-contiguous**, vectorized, memory efficient.
- Thích hợp cho computation heavy.

Data Structures & Libraries Benchmark - **Memory Management**

Python list:

- **Heterogeneous container**: list có thể chứa bất kỳ object nào (int, str, dict, thậm chí function).
  - Vì vậy mỗi phần tử không phải lưu giá trị trực tiếp, mà list lưu con trỏ (pointer) tới object ở vùng nhớ khác.
- **Dynamic resizing**: list cần cấp phát dư thừa (over-allocation) để hỗ trợ append/extend nhanh hơn.
  - Ví dụ: list 10 phần tử có thể allocate 12–14 slot. Điều này gây lãng phí bộ nhớ.
- **Object overhead**: mỗi phần tử trong list (vd int) là một Python object (PyObject) → ngoài giá trị còn có metadata (type info, ref count).
  - Một int Python có thể tốn 28 bytes thay vì 4 bytes như C int.

Data Structures & Libraries Benchmark - **Memory Management**

NumPy array:

- **Homogeneous container**: tất cả phần tử cùng type (vd float64).
  - NumPy chỉ lưu raw data (giống như C array).
- **Contiguous memory block**: dữ liệu lưu liên tục trong RAM → ít overhead, CPU cache friendly.
- **Fixed-size dtype**: mỗi phần tử có kích thước cố định (int32, float64, …).
  - Không cần con trỏ, không cần metadata cho từng phần tử.
- **Vectorization**: nhiều phép toán chạy ở C-level, không cần loop Python → vừa nhanh vừa memory efficient.

Data Structures & Libraries Benchmark - **Pandas**

```python
import pandas as pd

df = pd.DataFrame({"num": range(10_000_000)})
# Vectorized operation (underneath uses NumPy)
df["squared"] = df["num"] ** 2
```

- Dựa trên NumPy, có overhead DataFrame.
- Dễ thao tác, nhưng apply/iterrows chậm.
- Ưu tiên dùng vectorized.

Data Structures & Libraries Benchmark - **Polars**

```python
import polars as pl

df = pl.DataFrame({"num": range(10_000_000)})
df = df.with_columns((pl.col("num") ** 2).alias("squared"))
```

- **Rust-based**, lazy execution.
- Performance vượt trội trên dataset lớn.

Data Structures & Libraries Benchmark - PySpark

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

# Tạo SparkSession local (chạy trên CPU laptop của bạn)
spark = SparkSession.builder \
    .appName("SquareDemo") \
    .master("local[*]") \   # "*" = dùng tất cả core CPU
    .getOrCreate()

df = spark.range(10_000_000).withColumn("squared", col("id") ** 2)
df.show(5)
```

- **Distributed**, **horizontal scale**.
- Overhead khởi động lớn → phù hợp big data.

Downcast Numeric Columns - Memory Optimization

Mục tiêu: Giảm memory usage bằng cách hạ thấp dtype mà vẫn giữ đúng giá trị.

Concept:

- Mặc định Pandas dùng **int64** và **float64** (8 bytes) → chiếm nhiều RAM.
- Nếu dữ liệu nhỏ hơn range, có thể downcast xuống **int32** (4 bytes), **int16**, **float32**…

Dtype	Bit	Byte	Range signed (có âm)
int32	32	4	-2_147_483_648 → 2_147_483_647
int64	64	8	-9_223_372_036_854_775_808 → 9_223_372_036_854_775_807

Categorical - Memory Optimization

Mục tiêu: Giảm memory cho **string/object columns**.

Concept:

- String/object column chiếm nhiều memory vì lưu full string cho mỗi row.
  - Python 3 dùng Unicode cho str → mỗi **ký tự** có thể chiếm **1–4 bytes** tùy encoding nội bộ (**UTF-8** / **UCS-2** / **UCS-4**) + thêm 37 - 49 bytes overhead (pointer, refcount, length, hash, …)
- category → chỉ lưu **integer code + mapping** → tiết kiệm RAM & Disk lớn.

Loại	Memory per value
Python str (object)	1–4 bytes/char + ~37–49 bytes overhead
Pandas category	1–4 bytes/int code + 1 lần lưu string unique

Pandas - Memory Optimization

- Ưu tiên dùng **Vectorization** (tận dụng C backend) khi được, tránh lạm dụng **.apply()**
  - Vd: ưu tiên dùng vectorize bằng **str** accessor:
    - **df['col'] = df['col'].str.lower().str.strip()**
  - thay vì
    - .apply(lambda x: x.strip().lower())

- Sử dụng **inplace=True** khi điều chỉnh dataframe để **giảm memory copy**
  - Vd: nên dùng:
    - **df.drop(columns=['col'], inplace=True)**
  - thay vì
    - df = df.drop(columns=['B'])

Generator expressions - Memory Optimization

- Ưu tiên dùng **Generator expressions** để **giảm memory** cho sequence dài
  - Vd: generator không tạo list, memory thấp
    - **total = sum(x*x for x in range(10_000_000))**
  - thay vì tạo list toàn bộ 10 triệu số bình phương → tốn memory
    - total = sum([x*x for x in range(10_000_000)])

Python – High-level, Dynamic

Đặc trưng:

- Thông dịch bởi **CPython** Interpreter (không biên dịch xuống native machine code).
- **Dynamic typing** → linh hoạt nhưng tốn overhead runtime.
- Thư viện phong phú (NumPy, Pandas, TensorFlow…).

Performance:

- Chậm hơn C/C++/Rust do:
  - Mỗi toán tử là một function call qua CPython API.
  - Bộ nhớ quản lý bằng garbage collector + reference counting.
- Phù hợp prototyping, scripting, ML pipeline orchestration.

Cython – Python with C Speed

Đặc trưng:

- Viết code gần giống Python, thêm type annotation.
- Compiler: **dịch sang C code** → rồi compile bằng gcc/clang.
- Cho phép gọi thẳng thư viện C (NumPy, BLAS, OpenCV…).

Ưu điểm:

- Giữ cú pháp Python nhưng đạt **hiệu năng C-level**.
- Chạy tốt cho CPU-bound loop, numerical computing.

Nhược điểm:

- Vẫn phụ thuộc **Python runtime** (không độc lập như Rust).
- Cần compile, khó deploy hơn thuần Python.

File extensions

File	Loại	Sinh ra khi nào	Chứa gì
.py	Source	Do developer viết	Code Python (text)
.pyc	Bytecode	Khi Python chạy `.py` thì compile sang `.pyc` (lưu cache trong `__pycache__`)	Bytecode (interpreter đọc được, không phải machine code)
.pyd	Binary (DLL cho Windows)	Khi build C extension hoặc compile bằng Cython/setuptools	Machine code (nạp trực tiếp bởi CPython)
.so	Binary (Shared Object cho Linux/Mac)	Tương tự `.pyd` nhưng trên Linux/Mac	Machine code

Rust – Systems Language, Safe & Fast

Đặc trưng:

- Ngôn ngữ **system-level** (thay thế C/C++).
- **Ahead-of-Time (AOT) compilation** → binary machine code tối ưu.
- **Borrow checker** → memory safety mà không cần GC.

Ưu điểm:

- Hiệu năng ngang C/C++, tối ưu cho backend/service/data engine.
- Có thể build shared library để Python gọi qua pyo3 / maturin.

Nhược điểm:

- Cú pháp phức tạp, học khó hơn Python/Cython.
- Ecosystem cho ML/data chưa phong phú bằng Python.

Compiler – Python vs. Cython vs. Rust

1. Quy trình khi chạy một file Python bằng **CPython** (trình thông dịch mặc định)**:**

- **Parser (phân tích cú pháp)**: Source code .py được phân tích cú pháp thành **Abstract Syntax Tree (AST)**.
- **Bytecode Compiler**: AST được dịch sang **bytecode** (mã trung gian, không phải C code). Bytecode là thứ bạn thấy trong file **.pyc** trong thư mục **__pycache__**.
- **Interpreter / Virtual Machine**: CPython VM (một vòng lặp C gọi là **eval loop**) đọc từng lệnh bytecode và thực thi nó bằng các hàm viết bằng C trong CPython runtime, mỗi operation tốn thêm nhiều bước: parse, dispatch, gọi hàm C, check type, allocate object.

Ví dụ:

a = b + c

- Trong C: dịch thẳng thành 1–2 lệnh máy (ADD).
- Trong Python:
  - Dịch thành bytecode: LOAD_FAST b, LOAD_FAST c, BINARY_ADD, STORE_FAST a.
  - Interpreter đọc từng opcode, gọi hàm C để xử lý → overhead rất lớn.

Compiler – Python vs. Cython vs. Rust

2. C:

- Code dịch **trực tiếp** từ source sang machine code bằng compiler (gcc, clang, rustc).

  → CPU chạy **native instructions** không qua bước trung gian.
- Compiler dịch thành vài lệnh máy add / cmp / jmp
- Không có overhead object, type checking

3. Cython

- Cho phép viết code Python-like nhưng dịch sang C code, sau đó compile bằng gcc/clang.

  → Nhờ đó bỏ qua overhead của interpreter ở loop & arithmetic.

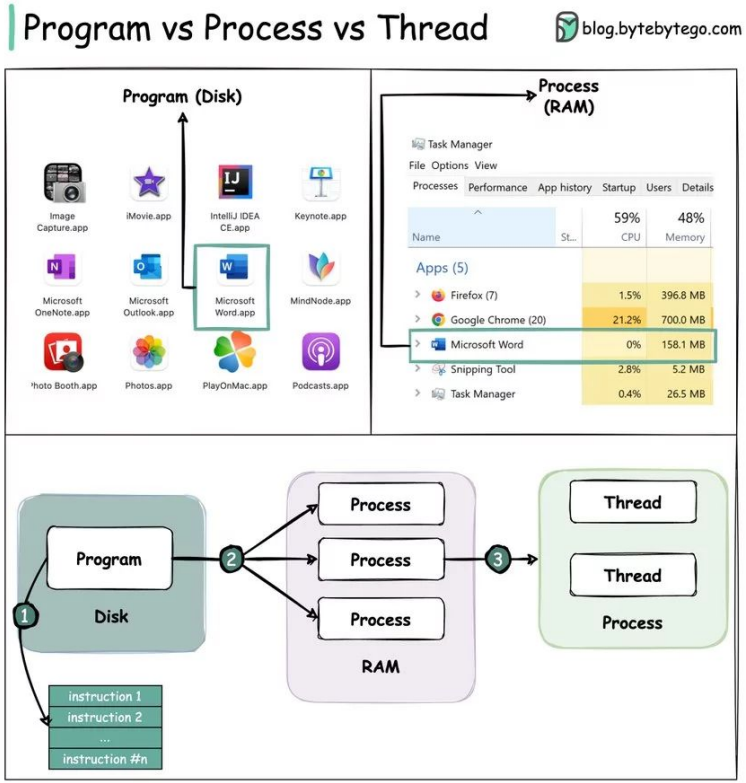
4. Rust

- Biên dịch thẳng sang machine code, kiểm soát memory & concurrency tốt hơn.

Python vs. Cython vs. Rust

- **Python**: prototyping, scripting, ML pipelines, glue code.
- **Cython**: khi đã có Python code và cần tối ưu loop/numerical core.
- **Rust**: khi cần system-level speed, memory safety, service backend.

Thread vs. Process



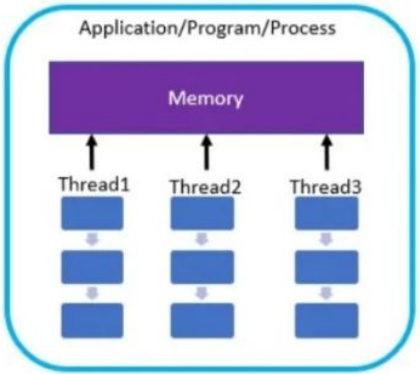
Process (chương trình đang chạy):

- Có vùng nhớ riêng biệt (code, stack, heap, data).
- Khi OS tạo process mới → cấp phát bộ nhớ độc lập.
- Một process có thể chứa nhiều thread.
- Ví dụ: mở 2 file Word → 2 process độc lập.

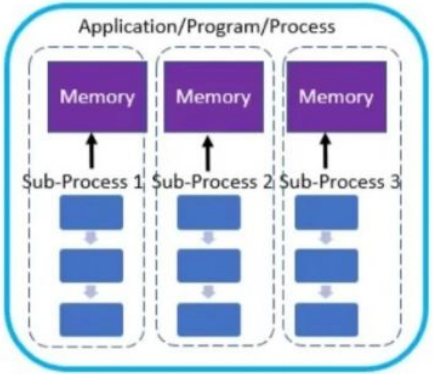
Thread (đơn vị nhỏ hơn process):

- Các thread chạy trong cùng process → share:
  - Code segment.
  - Heap (dữ liệu).
- Nhưng mỗi thread có stack riêng (biến cục bộ, call stack).
- Ví dụ: trong Word:
  - 1 thread xử lý kiểm tra chính tả,
  - 1 thread xử lý auto-save.

Multi-Thread vs. Multi-Process



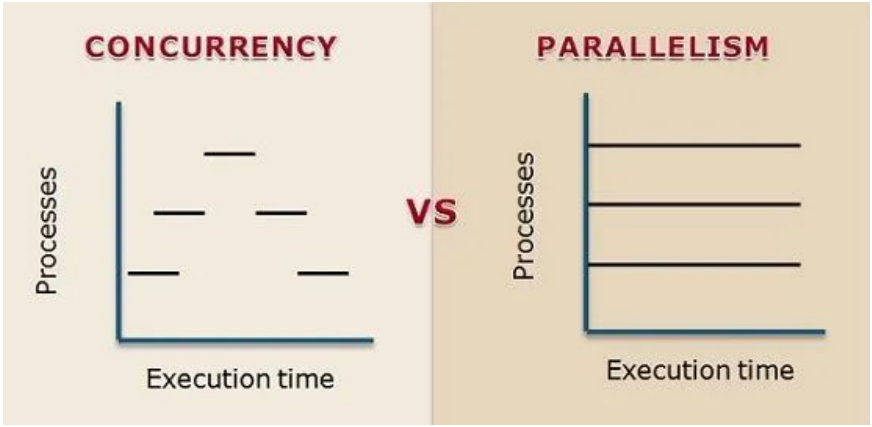
Multi-Threading



Multi-processing

- Mỗi **process** có không gian bộ nhớ riêng. Các **thread** thuộc cùng một tiến trình sẽ chia sẻ cùng một bộ nhớ.
    - Vì vậy khi lập trình **multithread** thì cần lưu ý xử lý vấn đề xung đột bộ nhớ.
- Một **process** đều heavy weight, nó cần nhiều thời gian để tạo và chấm dứt.
- Chuyển đổi ngữ cảnh giữa các **process** tốn kém, giao tiếp giữa các **thread** thì nhanh hơn.

Concurrency vs. Parallelism



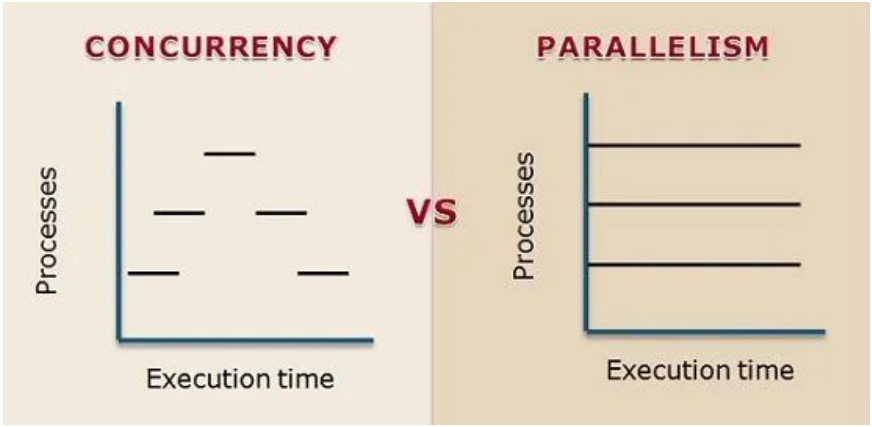
Concurrency (chạy đồng thời):

- Nhiều task cùng tiến triển, nhưng có thể chỉ 1 core, dùng time-slicing & context switching
- Mục tiêu: tăng **responsiveness**.
- Ví dụ minh họa: 1 nhân viên chạy qua lại giữa nhiều bàn

Parallelism (chạy song song):

- Nhiều task chạy thật sự cùng lúc trên nhiều core.
- Mục tiêu: tăng **throughput**.
- Ví dụ minh họa: nhiều nhân viên phục vụ nhiều bàn cùng lúc

Concurrency vs. Parallelism



Lịch sử:

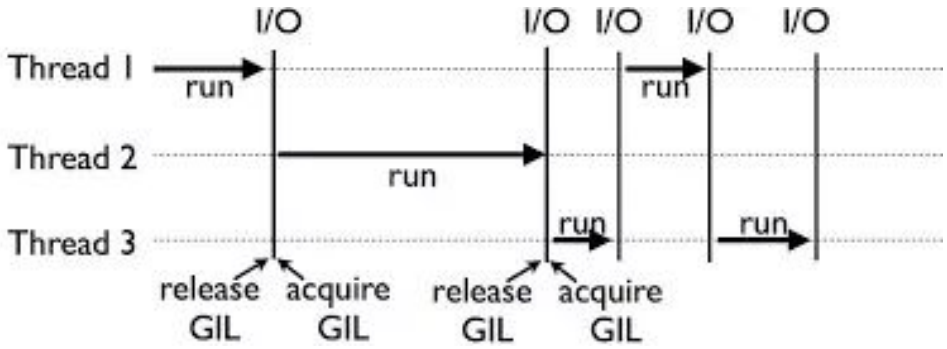
- Thời CPU 1 core → **Concurrency** = giải pháp phần mềm (time-slicing, context switching).
- Song song thật sự (parallelism) chưa tồn tại.

Hiện tại:

- CPU multicore phổ biến → **Parallelism** = giải pháp phần cứng (nhiều core chạy đồng thời).
- Tuy nhiên:
  - **Số task >> số core** (ví dụ: web server 10k request, CPU chỉ 16 core).
  - **Nhiều tác vụ I/O** → không cần core mạnh, cần **concurrency** để xoay vòng.

Hệ thống hiện đại = Concurrency + Parallelism
(quán đông khách → cần cả nhiều nhân viên + mỗi nhân viên lo chăm sóc nhiều bàn)

Global Interpreter Lock (GIL) in Python



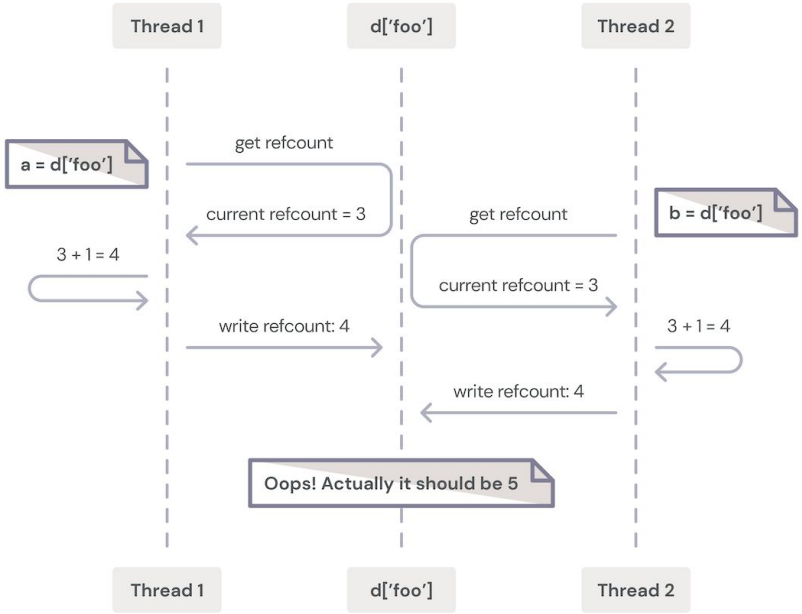
- **GIL**: một cơ chế khóa toàn cục trong Python (khi dùng CPython).
- Cho phép **chỉ 1 thread chạy Python bytecode** tại **1 thời điểm.**
- Dù máy có nhiều core (4, 8, 16), **Python multithreading** vẫn không tận dụng hết CPU.
- GIL là mutex (lock), dùng để quản lý truy cập bộ nhớ trong interpreter.
- Khác với Java/C++ → nơi multithreading có thể chạy song song thật sự.

Reference Counting - Memory Management Technique in Python

- Mỗi object trong bộ nhớ sẽ giữ một bộ đếm tham chiếu (**reference count**).
- Bộ đếm này cho biết có bao nhiêu biến/đối tượng đang trỏ đến object đó.
- Khi **bộ đếm = 0** (không còn tham chiếu nào):

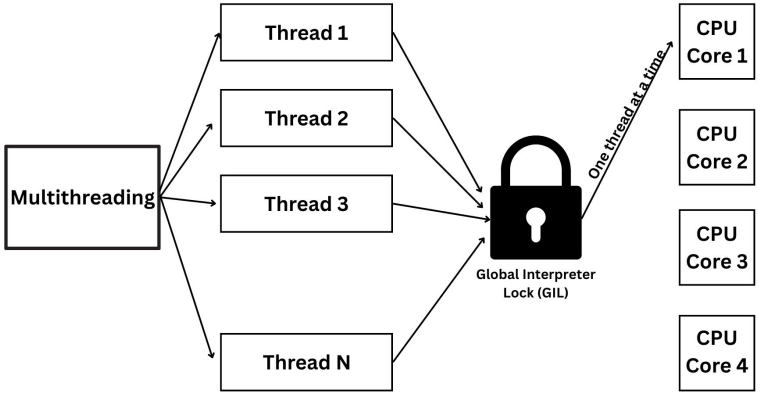
  → Object được giải phóng (**deallocated**) tự động.
- Cơ chế này giúp Python quản lý bộ nhớ hiệu quả,mà không cần **garbage collector** riêng biệt trong hầu hết các trường hợp.

Race Condition among Threads



- Code muốn tăng biến toàn cục (`counter`) từ 2 thread, nhưng **kết quả cuối cùng thường < tổng kỳ vọng**.
- Nguyên nhân: `counter += 1` không phải thao tác nguyên tử (**atomic operation**)
- Nếu 2 thread chạy cùng lúc, có thể chen ngang → kết quả sai.
- Bên trong, mỗi `+=` còn liên quan đến tăng giảm refcount của đối tượng int.
- **GIL** được thiết kế để:
  - Chỉ cho 1 thread chạy **Python bytecode tại 1 thời điểm**
  - **Ngăn concurrent modification** của refcount và các cấu trúc dữ liệu nội bộ
- **GIL**: deliberate design để tránh memory corruption / crash

Global Interpreter Lock (GIL) in Python



- **GIL** đảm bảo **thread safety** bên trong **CPython** bằng cách **khóa interpreter**, chỉ 1 thread chạy bytecode tại một thời điểm.
- **Đơn giản hóa quản lý bộ nhớ**: giảm rủi ro **race condition** với refcount và object nội bộ.
- **Giảm độ phức tạp khi viết code Python** an toàn và ổn định.
- **Hạn chế**: **giảm khả năng song song** thực sự (parallelism) trong chương trình multithread CPU-bound.

Global Interpreter Lock (GIL) in Python

Ảnh hưởng của GIL đến Multithreading

- **GIL** ảnh hưởng trực tiếp cách Python xử lý chương trình **multithread**.
- Đảm bảo **thread safety,** nhưng ngăn nhiều thread chạy Python bytecode song song
- Tác động của GIL không đồng đều, phụ thuộc loại task:
  - **I/O-bound tasks**: thao tác chờ hệ thống ngoài (disk, network) → **multithreading vẫn có lợi**.
  - **CPU-bound tasks**: tính toán liên tục, tiêu tốn CPU → **multithreading bị hạn chế nghiêm trọng**.

Multiprocessing - Alternative to working around the GIL

- Module **multiprocessing** cho phép Python tạo các **process độc lập** thay vì thread.
- Mỗi **process** có:
  - Không gian bộ nhớ riêng
  - Python interpreter riêng
  - **GIL riêng** → cho phép true parallelism trên máy nhiều core.
- Phù hợp với **CPU-bound tasks**:
  - Tính toán nặng (numerical simulations)
  - Chuyển đổi dữ liệu (data transformations)
  - Xử lý ảnh (image processing)
- Hạn chế:
  - **Tốn bộ nhớ** hơn thread, vì mỗi process có không gian riêng, không share memory.
  - Communication giữa process khó khăn hơn → phải dùng Queue, Pipe, Shared Memory…
  - Khởi tạo process chậm hơn thread, overhead lớn nếu tạo quá nhiều process nhỏ.

Async IO - Alternative to working around the GIL

- **Asyncio** là thư viện chuẩn của Python để viết **chương trình bất đồng bộ** (**asynchronous**).
- Sử dụng **coroutines** và **event loop** để quản lý các **task I/O**.
- Khi một async function **chờ I/O**, nó **nhường quyền điều khiển** → các task khác có thể chạy cùng lúc trong **một thread duy nhất**.
- **Không bị GIL cản trở**, vì chỉ có một thread duy nhất nhưng task chạy hiệu quả nhờ event loop.
- Thích hợp cho **I/O-bound workloads**:
    - Gửi API requests
    - Download file
    - Xử lý input/output thời gian thực

C extensions - Alternative to working around the GIL

- Một số thư viện Python (nhất là viết bằng **C** hoặc **Cython**) có thể **release GIL** trong quá trình tính toán.
- Khi GIL được nhường, các **task tốn thời gian** có thể chạy **song song với thread Python khác**.
- Tăng tốc **data processing, numerical computation**, hoặc **loop tính toán** với code biên dịch **không cần GIL**.
- Ví dụ thực tế:
  - **NumPy**: thực hiện phép toán trong **C**
  - **Cython**: cho phép viết hàm C-level trong Python, dùng **nogil: block** để **release GIL**.
  - **Pandas**: cũng nhường GIL trong một số tính toán nội bộ.

Alternative Python Interpreter - Alternative to working around the GIL

- Một số interpreter Python khác nhau có cách xử lý concurrency khác hoặc **loại bỏ GIL** hoàn toàn:
- Các ví dụ:
  - **Jython**: Python chạy trên **Java Virtual Machine** (**JVM**).
    - Dùng **Java threads, không có GIL**.
  - **IronPython**: Python trên **.NET runtime** (dùng cho dự án cần tích hợp với Java/.NET)
    - Loại bỏ GIL, cho phép **multithreading thực sự**.

GIL updates in each Python versions

Version	GIL còn không?	Điểm khác biệt
≤3.7	Có	GIL release mỗi 5ms hoặc khi I/O
3.8–3.10	Có	Cải thiện fairness khi thread switching
3.11	Có	Faster CPython, specialization, refcount khác → vẫn 1 thread chạy tại 1 thời điểm
3.12	Có (default)	Có build thử nghiệm no-GIL
3.13+	Optional	**Free-threaded CPython** (no-GIL) có thể build chính thức